First, look at AttrNames: it's a module with one single method, set_name_cache. Given a name and a value, set_name_cache defines a conventionally named constant with that value. For example, if you pass it the string "description", then it defines a constant named ATTR_description. AttrNames is somewhat similar to a *Clean Room (87)*; it only exists to store constants that represent the names of attributes.

Now move down to AttributeMethodCache. Its [] method takes the name of an attribute, and it returns an accessor to that attribute as an UnboundMethod. It also takes care of at least one important special case: attribute accessors are Ruby methods, but not all attributes names are valid Ruby method names. (You can read one counterexample in the comment to ReaderMethodCache#method_body above.) This code solves that problem by decoding the attribute name to an hexadecimal sequence and creating a conventional safe method name from it.

Once it has a safe name for the accessor, AttributeMethodCache#[] calls method_body to get a String of Code that defines the accessor's body, and it defines the accessor inside a Clean Room named simply @module. (We discussed additional arguments to method_eval, such as __FILE__ and __LINE__, in *The irb Example*, on page 144.) Finally, AttributeMethodCache#[] gets the newly created accessor method from the Clean Room and returns it as an UnboundMethod.

On subsequent calls, AttributeMethodCache#[] won't need to define the method anymore. Instead, @method_cache.compute_if_absent will store the result and return it automatically. This policy shaves some time off in cases where the same accessor is defined on multiple classes.

To close the loop, look back at the code of ReaderMethodCache. By overriding method_body and returning the String of Code for a read accessor, ReaderMethodCache turns the generic AttributeMethodCache into a cache for read accessors. As you might expect, there is also a WriterMethodCache class that takes care of write accessors.

Is your head spinning a little after this long explanation? Mine is. This example shows how deep and complex attribute methods have become, how many special cases they have covered, and how much they've changed since their simple beginnings. Now we can draw some general conclusions.

A Lesson Learned

Here is one question that developers often ask themselves: How many special cases should I cover in my code? On one extreme, you could always strive for code that is perfect right from the start and leaves no stones unturned. Let's